ROYAUME DU MAROC
*-*-*-*-*
HAUT COMMISSARIAT AU PLAN
*-*-*-*-*-*-*-*-*
Institut National
de Statistique et d'Economie
Appliquee
INSEA

# Projet de Fin d'Etudes

*****

**Conception et réalisation de la plateforme PartageVoisins**

Préparé par : **M. AFTYS Tarik**
**M. ou Mme FATIMI Youssef**
Sous la direction de : ***Mme. SAIDI Raja***

***Filière : DSE***

# Table des matières


**1 Présentation du projet** **3**
1.1 Modes de partage supportés . . . . . 3

**2 Analyse des besoins** **4**
2.1 Besoins fonctionnels . . . . . 4
2.2 Besoins non fonctionnels . . . . . 5

**3 Diagramme de cas d'utilisation** **6**
3.1 Vue d'ensemble par acteur . . . . . 6
3.2 Diagramme de cas d'utilisation . . . . . 7

**4 Diagrammes de séquence** **8**
4.1 DS1 — Réservation d'un objet . . . . . 8
4.2 DS2 — Publication d'un article . . . . . 9

**5 Diagramme de classes** **11**
5.1 Entités et leurs attributs clés . . . . . 11

**6 Diagrammes d'activité** **13**
6.1 Cycle de vie du statut d'une réservation . . . . . 13
6.1.1 Flux Prêt (`LOAN`) et Location (`RENTAL`) . . . . . 13
6.1.2 Flux Don (`DONATION`) . . . . . 13
6.1.3 Points de décision principaux . . . . . 15
6.2 Processus de modération d'un signalement . . . . . 15
6.2.1 Points de décision principaux . . . . . 17

**7 Architecture technique** **18**
7.1 Vue d'ensemble . . . . . 18

# Chapitre 1

# Présentation du projet

**PartageVoisins** est une plateforme communautaire web permettant aux habitants d'un même voisinage d'échanger, prêter, donner ou louer des objets du quotidien. L'objectif est double :

— **Social** : renforcer les liens de proximité entre voisins en favorisant l'entraide.
— **Écologique** : réduire la surconsommation et l'empreinte carbone en encourageant la réutilisation plutôt que l'achat neuf.

## 1.1 Modes de partage supportés

La plateforme supporte trois modes de partage distincts :

| Mode | Description |
|---|---|
| **Prêt** (`LOAN`) | L'objet est prêté temporairement et doit être rendu à l'issue de la période convenue. |
| **Don** (`DONATION`) | L'objet est cédé définitivement et gratuitement. Il n'y a pas de phase de retour. |
| **Location** (`RENTAL`) | L'objet est mis à disposition contre une compensation financière. Le paiement et la caution se font en main propre. |

Table 1.1 – Les trois modes de partage de PartageVoisins

# Chapitre 2

# Analyse des besoins

## 2.1 Besoins fonctionnels

| Réf. | Besoin |
|---|---|
| BF-01 | Les utilisateurs doivent pouvoir s'inscrire et s'authentifier. |
| BF-02 | Un utilisateur peut publier un objet (avec photos, localisation, catégorie, mode de partage). |
| BF-03 | Un utilisateur peut rechercher un objet par mot-clé, catégorie et proximité géographique. |
| BF-04 | Un utilisateur peut réserver un objet disponible via un formulaire. |
| BF-05 | La réservation déclenche automatiquement une conversation entre les deux parties. |
| BF-06 | Le propriétaire peut accepter ou refuser une demande de réservation. |
| BF-07 | Le propriétaire confirme la remise de l'objet (statut PRÊTÉE ou DONNÉE selon le mode). |
| BF-08 | Pour un prêt ou une location : l'emprunteur confirme la restitution, puis le propriétaire confirme la récupération (double confirmation). |
| BF-09 | Pour un don : le flux s'arrête après la remise. L'objet est marqué indisponible définitivement. |
| BF-10 | Le système calcule automatiquement le $CO_2$ économisé à chaque transaction. |
| BF-11 | Les deux parties peuvent s'évaluer mutuellement après la transaction. |
| BF-12 | Un utilisateur peut signaler un objet ou un utilisateur abusif. |
| BF-13 | L'administrateur peut modérer les annonces, les profils et les litiges. |
| BF-14 | Le système détecte automatiquement les retards (prêt/location uniquement) et envoie des alertes. |

TABLE 2.1 – Besoins fonctionnels

## 2.2 Besoins non fonctionnels

| Réf. | Besoin |
|---|---|
| BNF-01 | L'interface est disponible en 3 langues : Français, Anglais, Arabe. |
| BNF-02 | Les échanges de messages sont en temps réel (latence $<$ 500ms). |
| BNF-03 | Les données financières (prix, caution) sont stockées avec une précision à 2 décimales (`BigDecimal`). |
| BNF-04 | Les identifiants sont en UUID pour garantir l'unicité distribuée. |
| BNF-05 | Le stockage des images est externalisé sur Cloudinary. |
| BNF-06 | Les requêtes géospatiales sont optimisées via l'extension PostGIS. |
| BNF-07 | Les mots de passe sont hashés avec BCrypt. |
| BNF-08 | L'authentification est basée sur des tokens JWT (stateless). |

Table 2.2 – Besoins non fonctionnels

# Chapitre 3

# Diagramme de cas d'utilisation

## 3.1 Vue d'ensemble par acteur

**Visiteur** Utilisateur non authentifié pouvant consulter et rechercher des annonces, sans possibilité d'interagir avec la plateforme.

**Utilisateur Authentifié** Utilisateur connecté pouvant gérer son profil, consulter les objets, échanger des messages, évaluer d'autres utilisateurs et suivre son score $CO_2$. Les rôles d'Emprunteur et de Prêteur en héritent.

**Emprunteur** Utilisateur effectuant une demande de réservation. Il peut réserver un objet, confirmer son retour et annuler une demande en attente.

**Prêteur** Utilisateur publiant des objets. Il gère les annonces et les demandes de réservation, ainsi que la remise et la récupération des objets.

**Administrateur** Responsable de la modération de la plateforme, de la gestion des litiges, des statistiques et de la suspension des comptes en cas d'infraction.

**Système Automatisé** Acteur non humain exécutant des tâches automatiques : gestion des réservations expirées, envoi d'alertes, détection des objets inactifs et calcul des scores $CO_2$.

## 3.2 Diagramme de cas d'utilisation

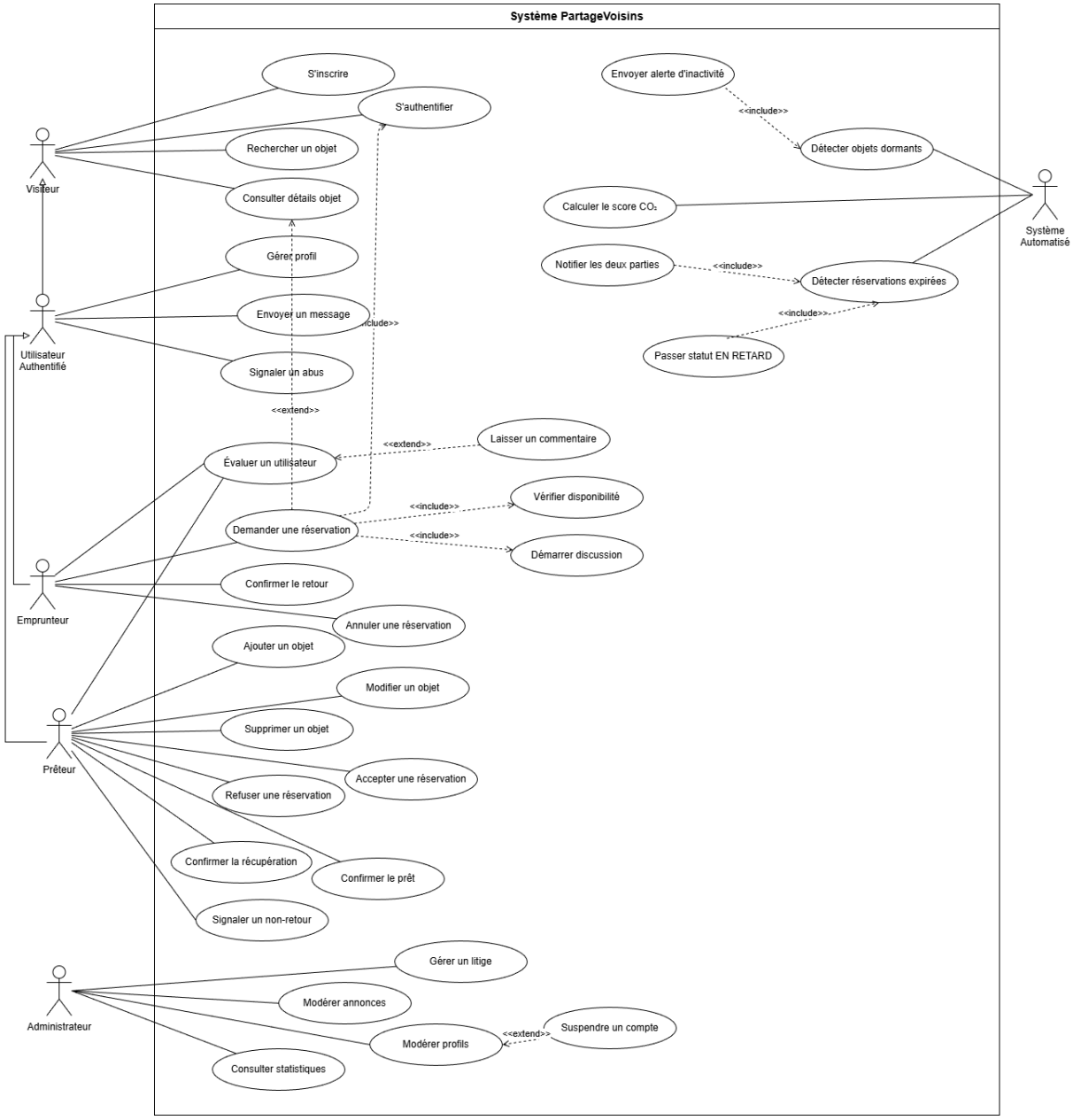



FIGURE 3.1 – Diagramme de cas d'utilisation — Les 5 acteurs et leurs interactions avec le système

# Chapitre 4

# Diagrammes de séquence

## 4.1 DS1 — Réservation d'un objet

**Participants** : Emprunteur, Frontend, Backend, BDD, Prêteur

**Phases couvertes** :

- **Phase 1 — Demande** : Vérification disponibilité → Création Réservation + Conversation simultanément → Notification du Prêteur.
- **Phase 2 — Décision** : Bloc `alt` → Acceptation (statut ACCEPTÉE, `isAvailable = false`) ou Refus (statut REFUSÉE + motif).

**Statuts traversés** : `EN_ATTENTE` → `ACCEPTÉE` | `REFUSÉE`

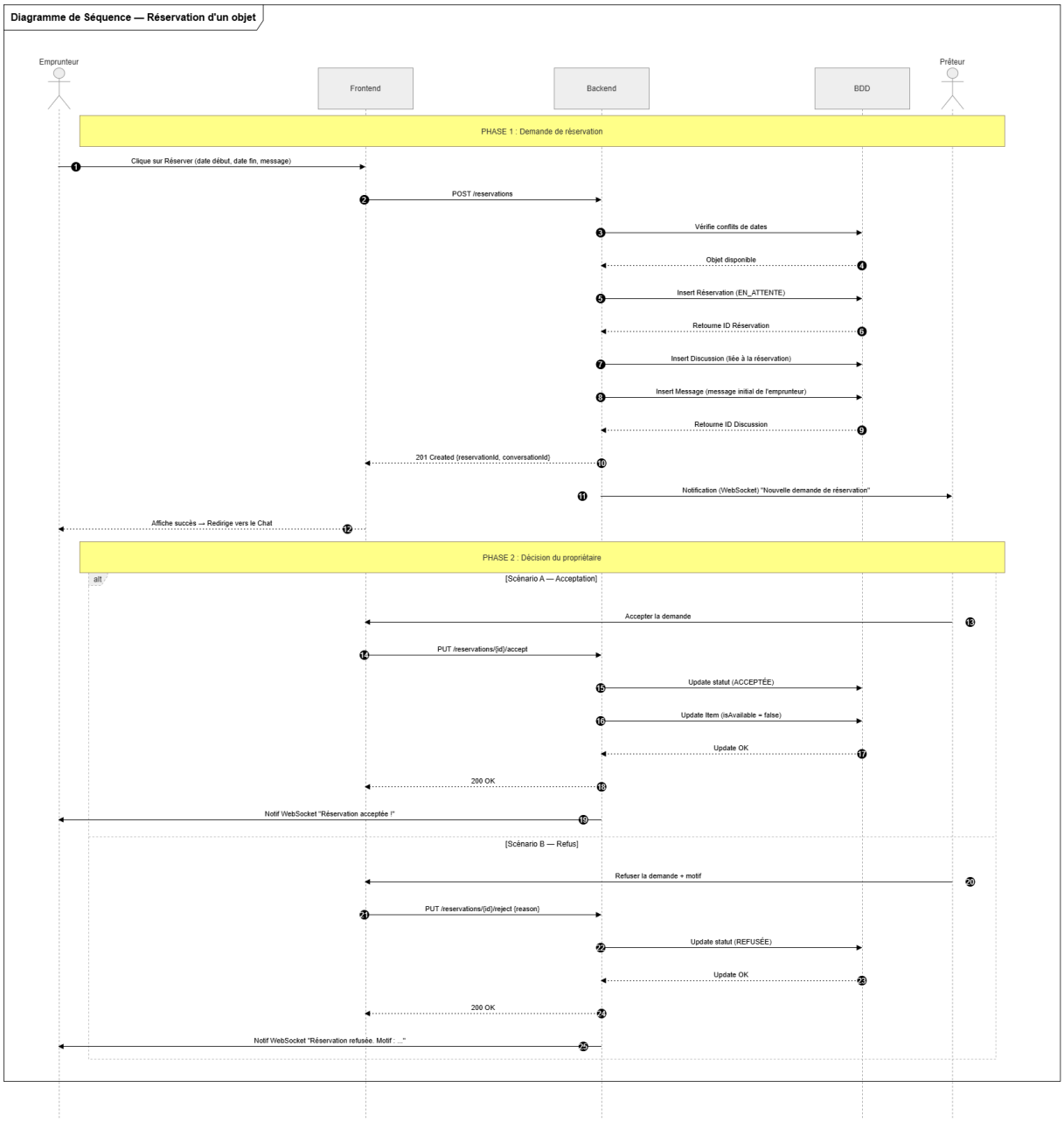



FIGURE 4.1 – DS1 — Diagramme de séquence : Réservation d'un objet

## 4.2 DS2 — Publication d'un article

**Participants** : Prêteur, Frontend, Backend, Cloudinary, BDD

**Phases couvertes** :

- **Phase 1 — Saisie** : Formulaire avec sélection du mode de partage (Prêt, Don ou Location) et localisation via carte Leaflet.
- **Phase 2 — Upload** : Envoi des photos vers Cloudinary → récupération des URLs sécurisées.
- **Phase 3 — Enregistrement** : Validation des données → Calcul $CO_2$ (Strategy Pattern) → `INSERT Item` avec coordonnées PostGIS → `INSERT Photos`.

**Points techniques mis en avant** :

— Upload Cloudinary avant la soumission du formulaire.

— Coordonnées stockées en type `GeometryPoint` (PostGIS).

— $CO_2$ calculé côté Backend selon la catégorie.

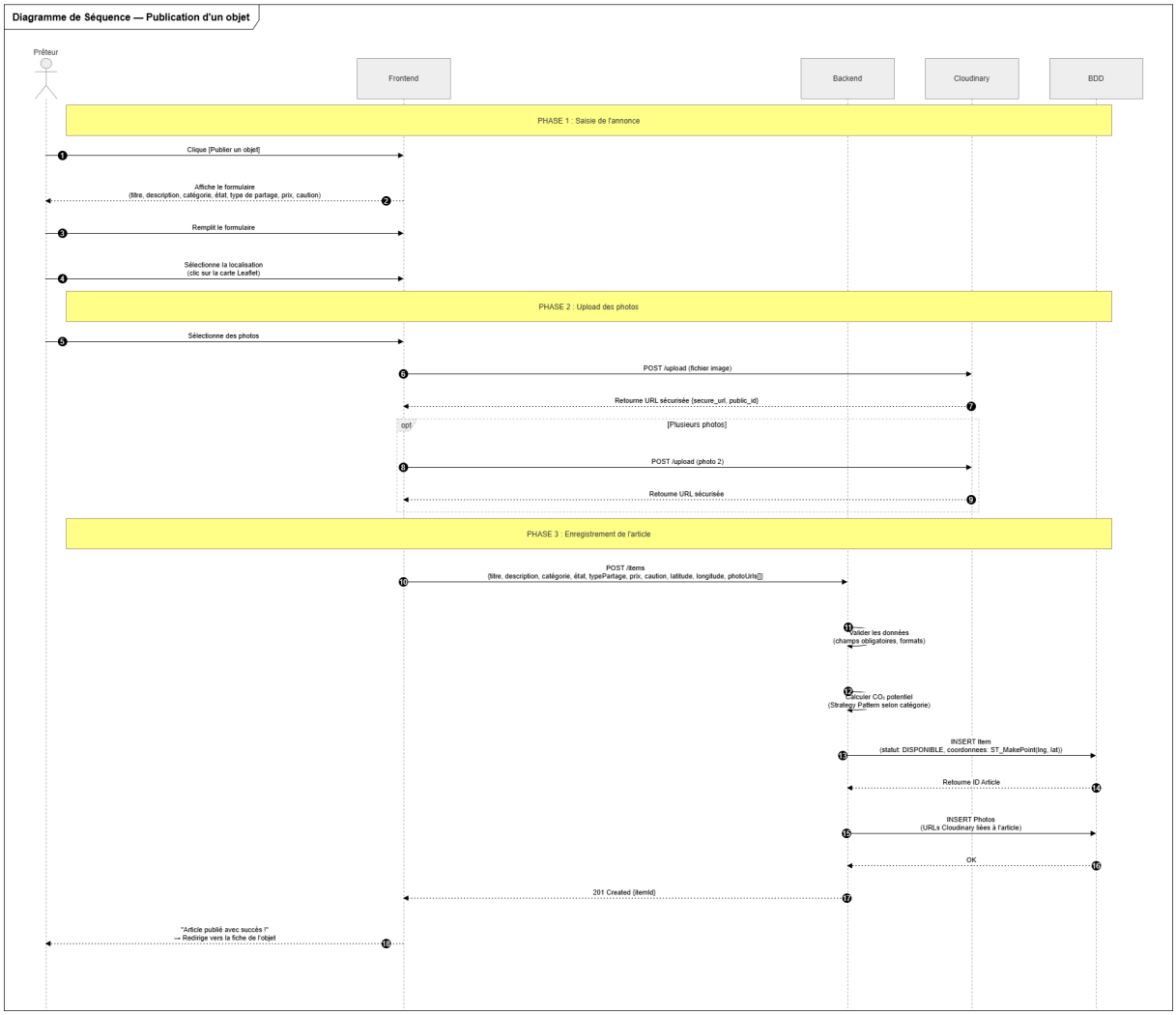



FIGURE 4.2 – DS2 — Diagramme de séquence : Publication d'un article

# Chapitre 5

# Diagramme de classes

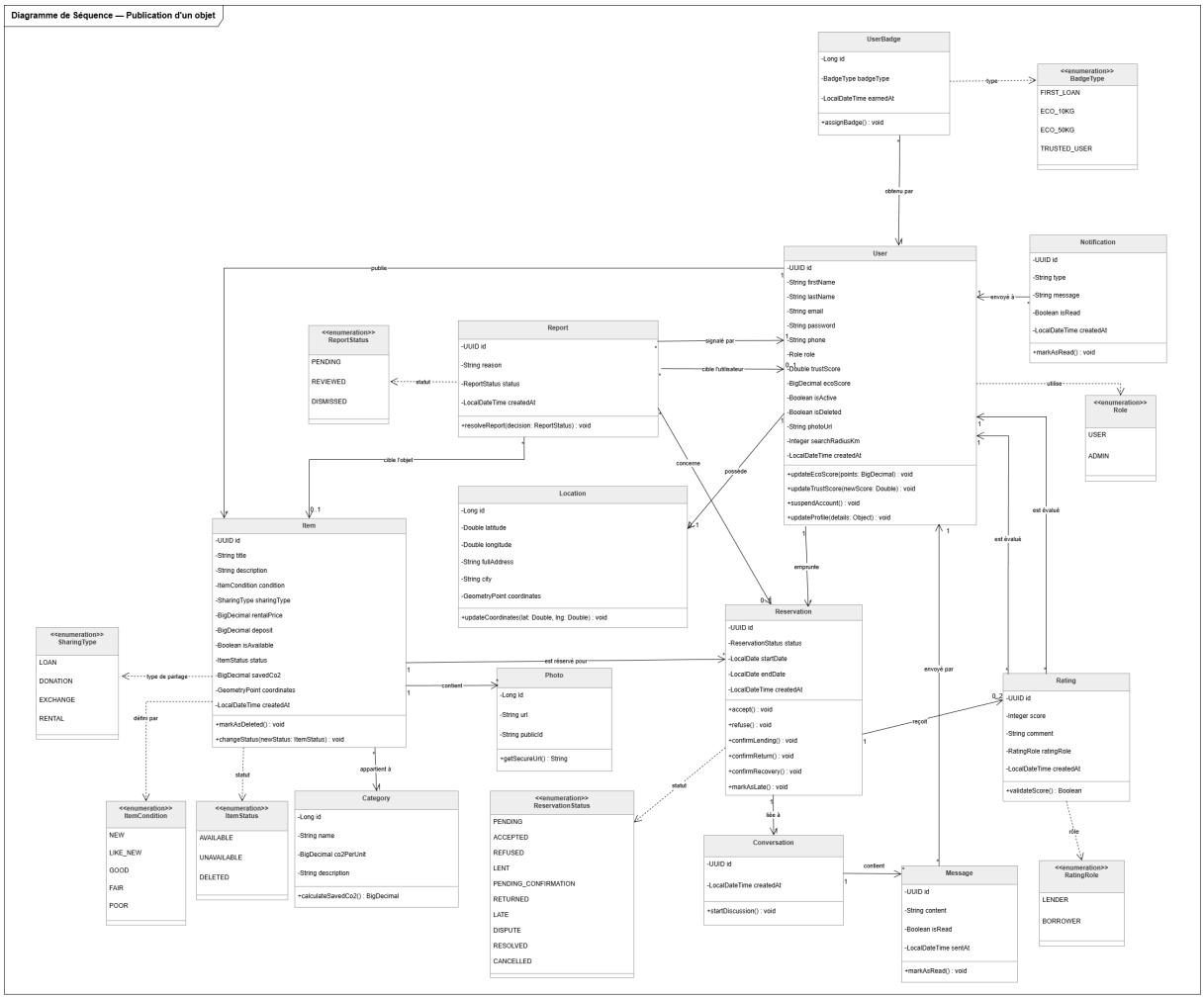



FIGURE 5.1 – Diagramme de classes — Les 12 entités, 8 énumérations et leurs relations

## 5.1 Entités et leurs attributs clés

| Entité | Attributs clés | Méthodes principales |
|---|---|---|
| **User** | id :UUID, email, role, trustScore, ecoScore, isActive | updateEcoScore(), updateTrustScore(), suspendAccount() |
| **Location** | id :Long, latitude, longitude, coordinates :GeometryPoint | updateCoordinates() |
| **Category** | id :Long, name, co2PerUnit :BigDecimal | calculateSavedCo2() |
| **Item** | id :UUID, title, condition, sharingType, rentalPrice, deposit, isAvailable, coordinates | markAsDeleted(), changeStatus() |
| **Photo** | id :Long, url, publicId | getSecureUrl() |
| **Reservation** | id :UUID, status, startDate, endDate | accept(), refuse(), confirmLending(), confirmReturn(), confirmRecovery(), markAsLate() |
| **Conversation** | id :UUID, createdAt | startDiscussion() |
| **Message** | id :UUID, content, isRead, sentAt | markAsRead() |
| **Rating** | id :UUID, score :Integer, comment, ratingRole | validateScore() |
| **Report** | id :UUID, reason, status, reservationId | resolveReport() |
| **Notification** | id :UUID, type, message, isRead | markAsRead() |
| **UserBadge** | id :Long, badgeType, earnedAt | assignBadge() |

TABLE 5.1 – Entités du système

# Chapitre 6

# Diagrammes d'activité

## 6.1 Cycle de vie du statut d'une réservation

Le cycle de vie d'une réservation varie selon le mode de partage choisi lors de la publication de l'objet.

### 6.1.1 Flux Prêt (`LOAN`) et Location (`RENTAL`)

Une réservation débute à l'état `PENDING` lorsque l'emprunteur soumet sa demande. Le propriétaire peut l'accepter (`ACCEPTED`) ou la refuser (`REFUSED`), et l'emprunteur peut l'annuler à tout moment (`CANCELLED`).

Une fois acceptée, le propriétaire remet l'objet et confirme le prêt ou la mise à disposition : la réservation passe à `LENT`. L'emprunteur initie ensuite le retour (`PENDING_CONFIRMATION`), que le propriétaire doit confirmer pour clôturer la transaction à l'état `RETURNED`.

En cas de non-retour après la date prévue, le système détecte automatiquement le retard (`LATE`). Le propriétaire peut alors signaler un litige (`DISPUTE`), qui sera traité par un administrateur jusqu'à sa résolution (`RESOLVED`).

### 6.1.2 Flux Don (`DONATION`)

Pour un don, le flux est plus court car aucune restitution n'est attendue. La demande passe par les mêmes états initiaux (`PENDING`, `ACCEPTED`, `REFUSED`, `CANCELLED`), mais une fois le don confirmé en main propre par le donneur, la réservation atteint l'état terminal `GIVEN`.

À ce stade, l'objet est définitivement marqué `UNAVAILABLE` car il a changé de propriétaire. Les états `LENT`, `LATE`, `DISPUTE` et `PENDING_CONFIRMATION` ne s'appliquent pas au mode don.

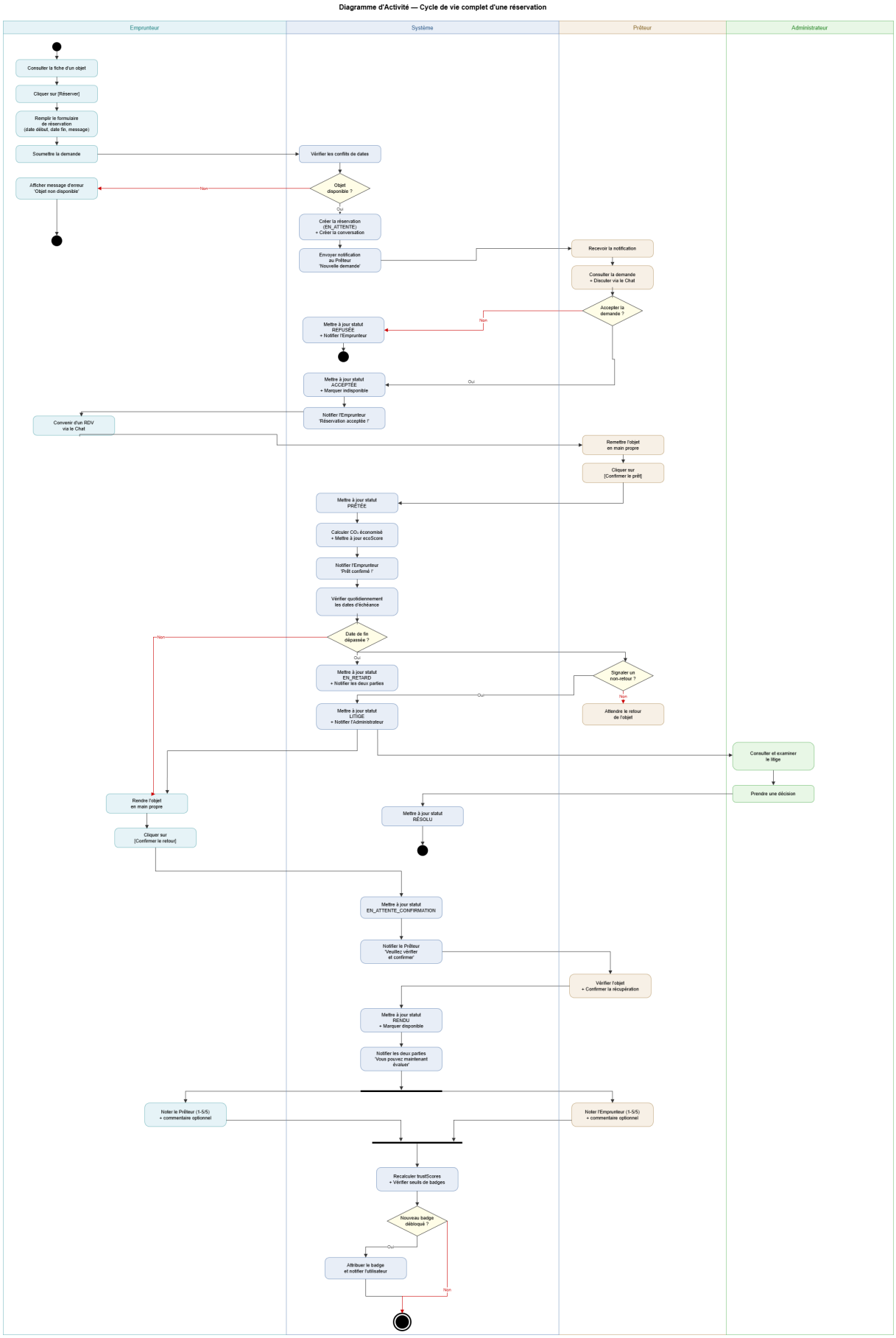



FIGURE 6.1 – Diagramme d'activité : Cycle de vie complet d'une réservation (4 swim lanes)

### 6.1.3 Points de décision principaux

| Point de décision | Chemins possibles |
|---|---|
| Objet disponible ? | Oui → Créer réservation / Non → Erreur → Fin |
| Propriétaire accepte ? | Oui → ACCEPTÉE / Non → REFUSÉE → Fin |
| Mode = Don ? | Oui → GIVEN → Évaluation → Fin / Non → continuer (prêt/location) |
| Date de fin dépassée ? | Oui → EN_RETARD / Non → Retour normal |
| Signalement non-retour ? | Oui → LITIGE → Admin / Non → Attendre |
| Nouveau badge ? | Oui → Attribuer / Non → Fin |

TABLE 6.1 – Points de décision du DA1

## 6.2 Processus de modération d'un signalement

**Swim Lanes** : Utilisateur | Système | Administrateur

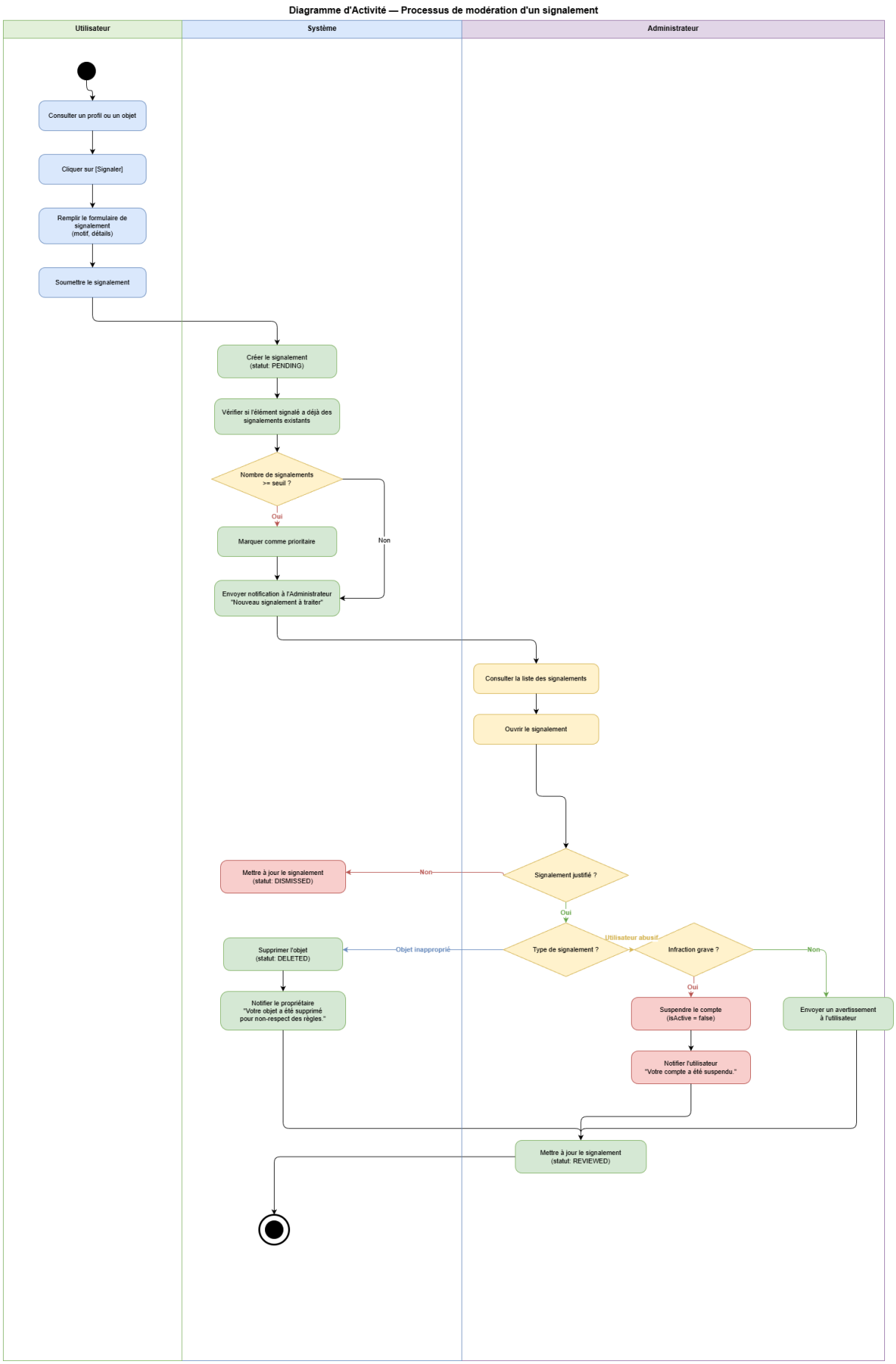



FIGURE 6.2 – DA2 — Diagramme d'activité : Processus de modération d'un signalement (3 swim lanes)

### 6.2.1 Points de décision principaux

| **Point de décision** | **Chemins possibles** |
|---|---|
| Nb signalements $\geq$ seuil ? | Oui $\rightarrow$ Marquer prioritaire / Non $\rightarrow$ Continuer |
| Signalement justifié ? | Oui $\rightarrow$ Action / Non $\rightarrow$ DISMISSED |
| Type de signalement ? | Objet $\rightarrow$ Supprimer / Utilisateur $\rightarrow$ Décision |
| Infraction grave ? | Oui $\rightarrow$ Suspendre compte / Non $\rightarrow$ Avertissement |

TABLE 6.2 – Points de décision du DA2

# Chapitre 7

# Architecture technique

## 7.1 Vue d'ensemble

L'architecture suit un modèle **client-serveur** avec séparation stricte entre le frontend et le backend :

| **Couche** | **Technologie** | **Rôle** |
|---|---|---|
| Frontend | Next.js 21+ (App Router) | Rendu hybride SSR/CSR, TypeScript |
| Backend | Spring Boot 3+ | API REST, WebSocket STOMP, Sécurité |
| Base de données | PostgreSQL + PostGIS | Stockage relationnel + géospatial |
| Stockage média | Cloudinary | CDN pour les images |

TABLE 7.1 – Stack technique